BRAC UNIVERSITY
Department of Computer Science and Engineering
CSE220: Data Structures

A

Examination: Final
Duration: 1 Hour 40 Minutes

Semester: Summer 2025
Full Marks: 35
No. of Pages: 4

Name: (Please write in CAPITAL LETTERS)	ID:	Section:

✔ **Answer all questions. No washroom breaks. Understanding questions is part of the Exam.**
✔ At the end of the exam, put the question **paper** inside the answer script and **return both**.

Question 1: CO3 [6 Points]

Are the following statements True or False? Write the correct answer if False (In script)

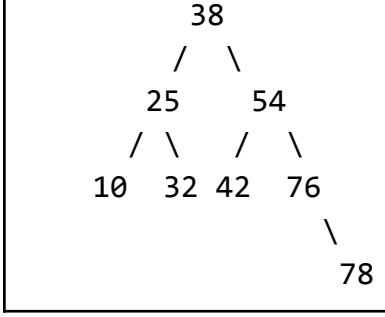
a) Breadth-First Search (BFS) can always visit all vertices of a directed graph in a single execution.
b) Every complete binary tree is not always a full binary tree.
c) The Worst-case Time Complexity for the insert operation of a heap and an unbalanced BST is the same.
d) In any directed, unweighted graph, the sum of all vertex degrees (in + out) is always even.
e) In a graph stored as adjacency lists, determining whether edge (u, v) exists takes O(1) time in the worst case.
f) Duplicate values are allowed in a HEAP.

Question 2: CO3 [5 Points]

Write the answers in your answer script

a) What are the space complexities of Adjacency Matrix and Adjacency List in graph representation? (0.5+0.5 mark)
b) MaxHeap represented as an array: `[X, 90, 80, 70, 40, 30, 20, 10]`
After executing extractMax() once, what will be the heap array? (1 mark)

i) `[X, 70, 80, 20, 40, 30, 10]`
ii) `[X, 80, 70, 20, 40, 30, 10]`
iii) `[X, 80, 40, 70, 10, 30, 20]`
iv) `[X, 10, 80, 70, 40, 30, 20]`



c) Write the PostOrder traversal sequence on the tree shown on the left. (1 mark)

d) The elements from the PostOrder traversal of the tree were sequentially inserted into a Min-Heap. Show the final tree. (2 mark)

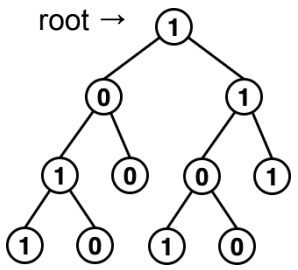
Question 3: CO4 [8 Points]

In a maze game, a player starts at the entry point of the maze, which is the root of a binary tree. Each node contains either a 0 or a 1. Each path from the root down to a leaf represents one possible way out through the maze. The player is given a sequence of 0s and 1s. They must check whether following that sequence will allow them to escape the maze by reaching a leaf node.

Write a function **findPath()** to help the player. This function will take the **root** of the binary tree, and the **sequence in an (int) array** to follow. The function will **return a string** "ESCAPED" if the sequence matches a complete path from the root to a leaf, "TRAPPED" if the sequence matches part of a path but ends at a non-leaf node, "LOST" if the sequence cannot be matched to any path in the tree.

Assume the `Node` class of the binary tree is already defined with the properties: `(int) elem, (Node) left, (Node) right.`

- You can take only one extra parameter or a helper function.
- You must solve this using recursion only.
- No extra data structures (array/linked list) or static/global variables are allowed.

Tree	Input	Returned String	Explanation
	`seq = {1,0,1,1}` `findPath(root, seq)`	ESCAPED	Matches path 1→0→1→1 Completely from root to leaf
	`seq = {1,1,0}` `findPath(root, seq)`	TRAPPED	Matches part of the path but ends at a non-leaf
	`seq = {1,0,0,1}` `findPath(root, seq)`	LOST	No such path in this tree

Python Notation	Java Notation

`def findPath(root, seq):`
`  #To do`
	`public static String findPath(Node root, int[] seq){`
	`  //To do`
	`}`

Question 4: CO1 [4 Points]

You are given an array of tasks, where each element represents the **urgency level** of that task and a deadline (number of days available). Your job is to complete the possible tasks within that deadline. However, there are a few rules:

- Each task takes **1 day** to complete.
- At each step, complete the task with the **highest urgency**.
- Ignore tasks left after the deadline.

The time complexity of your solution must be **less than $O(n^2)$. Assume that the classes MaxHeap and MinHeap are provided** with all necessary methods, including insert(value) and extract(), where extract() returns the removed element.

Caution: Using a Nested loop will result in $O(n^2)$ time complexity

You can create an Empty Heap using the following code	
Python Notation	**Java Notation**
`max_heap = MaxHeap()` `min_heap = MinHeap()`	`MaxHeap maxHeap = new MaxHeap()` `MinHeap minHeap = new MinHeap()`

Python Notation	Java Notation

`def urgent_task(tasks, deadline):`
`  #To do`
	`static void urgent_task(int[ ] tasks,int deadline){`
	`  //To do`
	`}`

Input	Output	Explanation

`tasks = [30, 95, 80, 25, 50]`
`deadline = 3`
	`Day 1 - Urgency 95`
	`Day 2 - Urgency 80`
	`Day 3 - Urgency 50`
	
	`Tasks completed: 3`
	`Tasks ignored: 2`
		Here, the deadline is 3 days and tasks 2, 3 and 5 have the highest urgency. So, they will be done in their respective urgency order. The rest of the tasks will be ignored.

Question – 5: CO2 [4 Points]

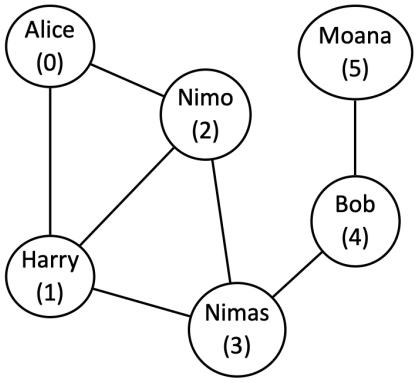
	0	1	2	3	4	5	6	7
0	0	6	0	0	7	0	0	0
1	6	0	4	0	8	0	0	3
2	0	4	0	2	0	1	8	0
3	0	0	2	0	9	0	0	0
4	7	8	0	9	0	0	0	0
5	0	0	1	0	0	0	9	0
6	0	0	8	0	0	9	0	0
7	0	3	0	0	0	0	0	0

The adjacency matrix shown above represents a weighted graph

a. Draw the Graph. [2 marks]
b. Show the traversal sequence by performing a special Depth First Search (DFS) traversal of the graph starting from vertex 2. The DFS is special because when a vertex has multiple unvisited neighbors, the traversal must visit the neighbors in ascending order of their edge weights, meaning the one with the smallest weight will be chosen to visit first. [2 mark]

Note: You do not need to write any code for this question, just show the simulation.

Question – 6: CO2 [8 Points]



You are given an **Undirected Unweighted Graph** representing a Facebook friendship network, where each vertex is a person and each edge is a friendship. Two people are said to have a mutual friend if there exists a third person who is directly connected to both of them. For example, Harry and Bob have one mutual friend, Nimas.

Your task is to write a function that finds the pair of distinct people that has the maximum number of mutual friends. If multiple pairs tie for the maximum, print any one of them. If no valid pair exists with at least one mutual friend, print the message, **"There are no mutual friends between any vertices."**

NOTE: The graph contains no self-loops or parallel edges. Vertex labels are mapped by a `mapper array (Type: String)`.

The function/method `maxMutual()` takes a graph `g represented in a 2D Matrix`, and the `mapper` as a 1D Array of Strings in the parameter.

Sample Input	Sample Output
mapper= ["Alice","Harry","Nimo","Nimas","Bob","Moana"] g = | 0 | 1 | 1 | 0 | 0 | 0 | | 1 | 0 | 1 | 1 | 0 | 0 | | 1 | 1 | 0 | 1 | 0 | 0 | | 0 | 1 | 1 | 0 | 1 | 0 | | 0 | 0 | 0 | 1 | 0 | 1 | | 0 | 0 | 0 | 0 | 1 | 0 |	Alice and Nimas have 2 mutual friend(s). **Explanation:** Alice and Nimas have 2 mutual friends, Harry and Nimo, which is the maximum in this graph.

Function/Method Skeletons:

Python Notation	Java Notation

`def maxMutual(g, mapper):`
`  # Write your code here`
	`static void maxMutual(int [][] g, String[] mapper) {`
	`  // Write your code here`
	`}`